PORTFOLIO · 2026

문영균,
iOS 개발자

필요했던 앱을 직접 만들어 출시하고 운영했습니다. 팀 프로젝트에서는 채팅 응답 처리와 데이터 동기화 문제를 확인하고 개선했습니다.

moongoon.cnu@gmail.com GitHub

출시와 운영	개인 iOS 앱
비동기 데이터	채팅 · 캐싱 · 동기화
AI 활용과 검토	기능 구현 · 운영 도구

프로젝트

01 · 격투 게임 정보 앱	1인 개발·출시·운영
02 · 토닥운	iOS 팀 개발 · 미출시
03 · 레츠톡	부스트캠프 웹·모바일 9기 · iOS 4인 팀
04 · 엑소스피어랩스	macOS 개발 인턴

01 · SOLO PROJECT

격투 게임 정보 앱

철권8 기술 탐색과 캐릭터별 메모 · 2025.03–현재

1인 개발·출시·운영

App Store GitHub

철권8을 플레이하며 캐릭터의 공격 동작과 입력 방법을 확인하려면 연습 모드로 이동해야 하는 불편을 느꼈습니다. 게임 화면을 전환하지 않고 스마트폰에서 기술 정보와 영상을 확인하고, 상대 캐릭터에 대응하는 방법을 메모할 수 있도록 앱을 만들었습니다.

UIKit + SwiftUI Combine Swift Concurrency Core Data Supabase CloudFlare XCTest

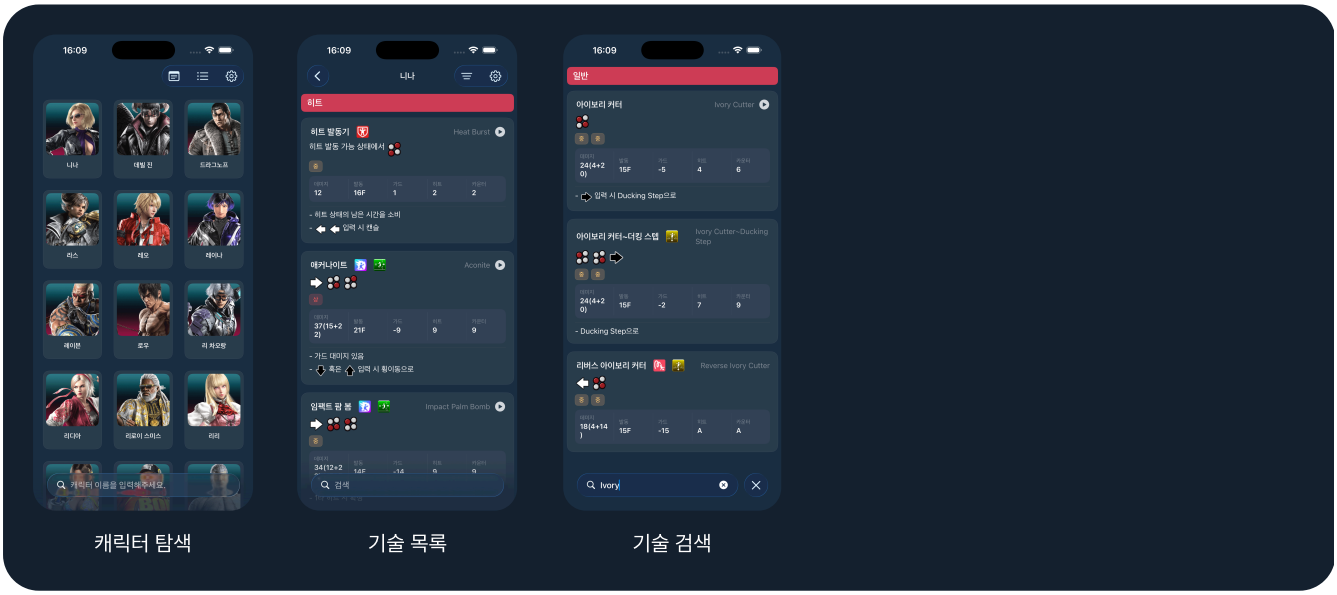
담당 범위

- 기획부터 iOS 앱 개발, App Store 출시와 운영까지 1인 담당
- Supabase 기술 데이터와 Cloudflare R2 영상 관리
- 사용자 요청에 따른 영어 지원과 데이터·이미지 로딩 개선

5.5K+
총 다운로드
2026.09 기준

900
월간 활성 사용자 달성
2026.09 당시 기록

1인
기획·개발·배포·운영
개인 프로젝트



격투 게임 정보 앱의 개발 화면 · 이미지를 누르면 원본을 볼 수 있습니다.

LOCALIZATION

01

한국어 원본 하나로 영어 지원

출시 후 해외 사용자에게 영어 지원 요청을 받았습니다. 게임 패치마다 기술 정보가 바뀌므로 한국어와 영어 데이터를 따로 관리하면 같은 변경을 두 번 반영해야 하고, 한쪽을 누락할 수 있다고 봤습니다.

01 한국어 원본 유지 | 02 커맨드 토큰 보호 | 03 설명 규칙 변환 | 04 커맨드 복원 · 영어 표시

제가 한 작업

- 한국어 기술 데이터(Move)는 유지하고, 앱에서 언어에 맞는 표시 모델(LocalizedMove)을 생성했습니다. 기술명은 기존 영문 필드를 활용하고, 섹션·판정·설명은 변환 규칙을 적용했습니다.
- 설명에 섞인 방향·버튼 커맨드를 토큰화해 임시 표식으로 보호했습니다. 나머지 문장에 고정 사전과 정규식 규칙을 적용한 뒤 원래 커맨드를 복원했습니다.
- 화면용 커맨드는 원래 표기를 유지하고, 영어식 방향·버튼 표기는 별도로 생성해 검색에 사용했습니다. 변환한 표시 모델은 메모리 캐시에 저장해 재사용했습니다.

달라진 점

별도 영문 데이터 수집 없이 한국어 원본으로 영어 표시를 지원했습니다. 영어 지원 업데이트 전후 전체 다운로드 중 한국 외 비중은 64.1%에서 약 86.2%로 증가했습니다.

실제 게임 화면에서 반복되는 표현을 확인

한국어와 영어 기술 설명을 실제 게임 화면에서 비교했습니다. '히트 상태에서', '입력 시 캔슬'처럼 반복되는 표현과 규칙을 확인해, 일부 예외를 제외하면 규칙 기반 변환이 가능하다고 판단했습니다. 영문 데이터를 별도로 수집하는 대신 한국어 원본에서 영어 표시를 만들기로 했습니다.

문장을 바꾸면서 커맨드는 보존

lp·rp 같은 버튼과 숫자 방향 입력은 일반 문장과 다르게 해석해야 합니다. 커맨드를 __CMD0__ 같은 임시 표식으로 바꿔 사전·정규식 규칙이 입력 표기를 변형하지 않도록 한 뒤, 문장 변환이 끝나면 원래 커맨드를 복원했습니다. 여러 줄과 괄호 안 설명에도 같은 흐름을 적용했습니다.

새 규칙이 기존 변환을 깨뜨리지 않도록 검증

규칙을 추가할 때 기존 규칙이 적용되지 않는 문제가 있었습니다. XCTest에 섹션 이름, 문맥 접두사, 고정 표현과 입력 시 캔슬 등의 사례를 추가했습니다. 공백·오탈자·빈 문자열·이미 영문인 입력도 확인합니다. 특수 커맨드와 새로운 표현은 규칙과 테스트를 보강해야 합니다.

이 경험에서 배운 점

기능을 추가할 때 당장의 구현뿐 아니라 패치 이후 관리할 범위까지 고려하게 됐습니다. 이후에는 해외 사용자도 사용할 수 있는지 함께 생각하게 됐습니다.

코드와 테스트에 있는 변환 예시

한국어 원본	영어 표시
히트	Heat
가드 데미지 있음	Chip damage on block
rp 입력 시 캔슬	rp to cancel

변경 전

64.1%

변경 후

약 86.2%

전체 다운로드 중 한국 외 국가의 비중

App Store Connect · 영어 지원 전: 최초 출시~2025.09.07 / 영어 지원 후: 2025.09.08~2026.10.08. 두 구간의 길이는 다르며, 영어 지원 후 구간은 전체 다운로드의 반올림 표시값으로 계산했습니다.

App Store Connect에서 확인한 영어 지원 전후의 다운로드 구성비입니다. 영어 지원만의 효과를 분리해 측정한 값은 아닙니다.

영어 지원 구현 PR #46 변환 회귀 테스트 추가 커밋

DATA CACHING

02 전체 데이터를 다시 받는 대신 버전으로 갱신 판단

기술 데이터는 게임 패치나 수정이 있을 때 바뀌는데도 앱 실행 때마다 서버에서 받아왔습니다. 실행 후 로딩이 발생했고, 네트워크가 없으면 기술 정보를 조회할 수 없었습니다.

01 버전 테이블 조회	02 서버 데이터가 갱신됐는지 확인	03 새 버전이면 갱신 / 같으면 저장 데이터 재사용	04 저장 데이터가 있으면 사용 / 없으면 서버에서 받아 저장

변경 전후 · 처리 방식과 책임

	변경 전	변경 후
데이터 조회	앱 실행마다 서버 전체 데이터 조회	유효한 로컬 값 재사용 · 없을 때 원격 조회
최신성 확인	전체 데이터를 받으면 최신 값 확보	별도 버전 테이블 비교 → 새 버전이면 캐시 무효화

제가 한 작업

- 기술 데이터를 Core Data에 저장하고, 서버의 데이터 버전을 로컬 버전과 비교하도록 구현했습니다.
- 서버 버전이 더 높으면 기존 기술 데이터 캐시를 비우고 저장 버전을 갱신합니다. 목록을 조회할 때 로컬 데이터가 없으면 Supabase에서 받아 저장합니다.
- 버전이 바뀌지 않고 유효한 로컬 데이터가 있으면 저장 데이터를 재사용합니다.

달라진 점

코드 측정 기준 원격 전체 조회는 0.41초, 버전 확인 후 로컬 데이터를 재사용하는 경로는 0.002초였습니다. 유효한 로컬 데이터가 있으면 오프라인에서도 기술 정보를 조회할 수 있습니다.

데이터 전체 비교 대신 버전 테이블을 만든 이유

캐싱 전에는 서버에서 항상 최신 데이터를 받아왔지만, 캐싱 이후에는 서버 데이터가 바뀌었는지 확인할 기준이 필요했습니다. 데이터 내용을 받아 비교하는 방법도 생각했지만, 매번 전체를 받으면 반복 다운로드를 줄이는 목적과 다르다고 판단했습니다. 별도 버전 관리 테이블을 만들고 그 값으로 변경 여부를 비교했습니다.

버전 판단과 데이터 조회의 책임을 구분

VersionManager는 서버 버전 비교와 캐시 무효화를 맡습니다. Repository는 목록 조회 시 로컬 값이 있으면 반환하고, 없으면 원격에서 받아 저장합니다. 화면은 데이터를 요청하고, 버전과 저장 위치에 따른 처리는 화면 밖에서 진행하도록 나눴습니다.

이 경험에서 배운 점

저장 자체보다 언제 데이터를 재사용하고 언제 갱신할지 기준을 정하는 일이 중요했습니다. 캐시가 유효한 상황에서는 네트워크 연결 없이도 기술 정보를 볼 수 있도록 제공 범위도 함께 고려했습니다.

변경 전	변경 후
0.41s	**0.002s**

데이터 확보 경로 변경 전후

원격 전체 조회와 버전 확인 후 로컬 재사용 경로를 비교한 코드 측정값입니다.

버전 기반 캐싱 PR #27

IMAGE CACHING & EGRESS

03

이미지 캐싱에서 Asset 전환까지, 로딩과 전송량을 함께 관리

앱 실행 때마다 캐릭터 이미지를 Supabase에서 받아와 표시가 늦어졌습니다. 캐싱을 적용한 뒤에도 사용자가 늘어난 시기에 무료 전송량 한도를 넘었습니다.

01	02	03	04
Asset 이미지 확인	있으면 바로 표시	없으면 메모리 → 디스크 확인	캐시도 없으면 원격 조회·저장

변경 전후 · 처리 방식과 책임

	변경 전	변경 후
캐싱 도입	이미지 필요 → 원격 다운로드	메모리 → 디스크 → 필요한 경우 원격 조회
Asset 전환	첫 사용자도 원격 이미지 다운로드 필요	캐릭터 Asset 우선 표시 → 없는 이미지 원격 경로

제가 한 작업

- NSCache와 FileManager로 메모리·디스크 캐싱을 직접 구현해 이미 받은 이미지를 재사용했습니다.
- 변경이 드물고 수량이 약 40개인 캐릭터 이미지를 앱 Asset에 포함했습니다.
- Asset에 이미지가 있으면 즉시 사용하고, 없는 경우에는 원격 이미지 캐싱 경로를 사용하도록 유지했습니다.

달라진 점

코드 측정 기준 원격 다운로드 1.1초와 기기 캐시 재사용 0.064초를 확인했습니다. 이후 약 40개 캐릭터 이미지의 서버 다운로드를 제거했고, Supabase에서 전송량 한도 문제가 해소된 것을 확인했습니다.

먼저 반복 다운로드를 줄임

메모리 캐시에서 이미지를 찾고, 없으면 기기 디스크에서 읽도록 구성했습니다. 디스크에서도 찾지 못할 때만 URLSession으로 다운로드해 메모리와 디스크에 저장했습니다. 같은 기기에서 다시 필요한 이미지는 저장한 값을 재사용하도록 했습니다.

캐싱 이후에도 한도를 넘은 이유를 검토

Supabase 전송량 한도를 넘었을 때 사용자 증가로 이미지 요청이 늘어난 상황을 검토했습니다. 다른 무료 이미지 호스팅을 사용하는 방법도 생각했지만, 원격에서 이미지를 받으면 첫 실행 사용자는 다운로드를 기다려야 했습니다. 바로 캐릭터 이미지를 보여주고 싶었고 수량도 많지 않아 Asset으로 옮기기로 했습니다.

측정값과 확인 결과

1.1초는 원격 다운로드, 0.064초는 기기 캐시 재사용 경로의 코드 측정값입니다. Asset 전환 후에는 Supabase에서 전송량 한도 문제가 해소된 것을 확인했습니다.

이 경험에서 배운 점

캐싱이 모든 로딩과 운영 비용 문제의 답은 아니었습니다. 데이터의 수량과 변경 빈도, 첫 실행의 표시 속도까지 함께 고려해 저장 방식을 선택하게 됐습니다.

RESPONSIBILITY & DEPENDENCIES

04

데이터 접근과 화면 객체 조립을 각 책임으로 분리

이전 프로젝트에서 MVC의 Controller가 비대해지는 문제를 경험했습니다. 이번 앱에서는 화면과 로직을 나누고자 했고, 로컬 캐싱을 추가하면서 원격·로컬 분기와 다음 화면의 객체 조립도 화면이 직접 맡지 않게 하고 싶었습니다.

01 캐릭터 선택

02 DIContainer에 화면 생성 요청

03 Repository·ViewModel·화면 조립

04 완성된 기술 목록 화면으로 전환

변경 전후 · 처리 방식과 책임

	변경 전	변경 후
데이터 접근	ViewModel → Supabase 통신 객체	ViewModel → Repository 인터페이스 → 저장소
객체 조립	화면이 필요한 조회 객체·ViewModel 생성	화면 → DIContainer 요청 → 조립된 다음 화면

제가 한 작업

- ViewController가 화면 표시를, ViewModel이 상태와 로직을 맡도록 MVVM을 적용했습니다.
- ViewModel의 서버 통신 객체 직접 의존을 Repository 인터페이스로 바꿔 필요한 데이터를 요청하도록 했습니다.
- 다음 화면에 필요한 Repository·ViewModel·ViewController의 생성과 연결을 DIContainer로 모았습니다.

달라진 점

데이터를 사용하는 계층은 저장 위치를 구분하지 않고 조회를 요청하게 됐습니다. 캐릭터 목록 화면은 선택한 캐릭터를 전달해 기술 목록 화면 생성을 요청하고, 객체의 생성과 연결은 DIContainer가 맡게 됐습니다.

데이터 접근의 변경 범위를 줄임

로컬 캐싱을 추가하면서 서버와 기기 데이터 중 어디에서 가져올지를 다른 계층이 알 필요가 없다고 판단했습니다. ViewModel은 Repository 인터페이스에 데이터를 요청하고, 실제 저장소 접근은 Repository가 맡도록 바꿨습니다. PR #11에서는 Controller가 직접 만들던 SupabaseManager와 ViewModel의 직접 서버 조회를 분리했습니다.

다음 화면의 객체 조립을 한곳으로 모음

캐릭터 목록에서 다음 기술 목록 화면으로 이동할 때는 다음 화면에 필요한 데이터 조회 객체와 ViewModel도 생성하고 연결해야 합니다. 이 조립 과정을 DIContainer로 모아 현재 화면이 직접 사용하지 않는 객체의 생성 방법까지 알지 않도록 했습니다. Controller에는 화면 생성을 요청할 DIContainer 의존이 남습니다.

이 경험에서 배운 점

데이터 접근을 수정할 때는 관련 계층을, 객체를 생성할 때는 DIContainer를 살펴보면 돼 수정할 위치를 찾기 쉬워졌습니다. 각 계층이 다른 계층의 구체 구현을 덜 알게 되면서, 변경이 다른 화면에 영향을 줄 수 있다는 걱정도 줄었습니다.

Repository·DIContainer 분리 PR #11

02 · YAPP · TEAM PROJECT

토닥운

사주 정보를 바탕으로 대화하는 AI 챗봇 · 2026.05–2026.08

iOS 팀 개발 · 미출시

GitHub

개인의 사주를 바탕으로 오늘의 운세와 추천 행동을 확인하고, 궁금한 점을 AI에 질문하는 서비스입니다.

온보딩 · 마이페이지 · AI 채팅과 공통 네트워크 · 인증 · UI 컴포넌트 개발에 참여했습니다.

SwiftUI | TCA | Tuist | Swift Concurrency | URLSession | Swift Testing

---

담당 범위

- 온보딩, 마이페이지, AI 채팅 기능 개발
- 공통 네트워크, 인증 처리와 공통 UI 컴포넌트 개발
- 팀 코드리뷰 전 사전 검토 스킬 제작·적용

2026년 10월 기준, App Store 정책 거절로 아직 출시하지 못했습니다.



팀에서 제작한 서비스 소개 이미지

CHAT DATA FLOW

01 서버 응답	02 NetworkCore / SSE	03 기능별 Client	04 TCA 상태	05 채팅 화면

SSE STREAMING

01

서버에서는 오던 AI 답변이 앱에서 보이지 않던 문제

AI 답변을 도착하는 대로 채팅에 표시하도록 개발했지만, 실제 앱에서는 답변이 나타나지 않았습니다. Swagger에서 직접 요청해 응답이 정상적으로 오는 것을 확인한 뒤 앱의 수신과 표시 과정을 조사했습니다.

01 Swagger 응답 확인	02 앱 수신 단계 조사	03 바이트에서 빈 줄 보존	04 SSE 이벤트 완성 → 채팅 표시

제가 한 작업

- 앱의 수신·표시 과정을 조사하고, SSE 이벤트를 구분하는 빈 줄이 유실되는 문제에 맞춰 파싱 방식을 보완했습니다.
- URLSession의 바이트 스트림을 읽어 줄 경계를 만들고, 비어 있는 줄도 파서에 전달하도록 변경했습니다.
- 파서가 빈 줄에서 하나의 이벤트를 완성하도록 유지해 응답을 채팅 상태에 전달했습니다.

달라진 점

이벤트 구분에 필요한 빈 줄을 파서까지 전달하도록 수신 로직을 수정했습니다. 파서는 이 경계에서 누적한 응답을 하나의 SSE 이벤트로 만들 수 있게 됐습니다.

응답 내용뿐 아니라 경계가 필요

SSE는 여러 줄을 하나의 이벤트로 모으고 빈 줄에서 이벤트를 끝냅니다. 내용이 도착해도 이 빈 줄이 파서에 전달되지 않으면 이벤트를 완성하는 시점을 놓칠 수 있습니다. 기존 파서는 빈 줄에서 누적한 데이터를 내보내므로 수신 단계에서 경계를 보존해야 했습니다.

바이트에서 줄을 만들 때도 빈 줄을 전달

줄 버퍼가 LF·CRLF·CR 줄바꿈을 처리하고, 줄 사이에 내용이 없을 때도 빈 문자열을 반환하도록 구현했습니다. SSEClient가 이 줄을 파서에 전달하고, 파서는 누적한 data를 하나의 이벤트로 만듭니다. 줄 경계와 잘못된 UTF-8 입력을 확인하는 테스트도 추가돼 있습니다.

이 경험에서 배운 점

서버 응답과 앱 동작을 비교해 문제를 조사할 구간부터 좁혔습니다. 익숙한 URLSession을 사용하더라도 데이터 형식에서 필요한 구분 정보가 읽기 과정에 보존되는지 확인해야 한다는 점을 배웠습니다.

SSE 이벤트 경계 보존 커밋

AI REVIEW WORKFLOW

02

AI 변경을 팀 코드리뷰 전에 점검하는 사전 검토 스킬

AI에 프로젝트 규칙을 제공해도 생성된 코드가 이를 지키지 않는 경우가 있었습니다. 팀 코드리뷰로 넘기기 전에 변경을 다시 확인하고 필요한 수정을 반영하는 과정이 필요했습니다.

01 Draft PR 생성	02 관점별 사전 검토	03 지적 근거 재확인·수정	04 관련 검증 → 팀 코드리뷰

제가 한 작업

- CodeRabbit 리뷰가 실행되지 않는 Draft PR 단계에서 사용할 사전 검토 스킬을 만들었습니다.
- AI가 아키텍처·테스트·빌드 등 서로 다른 관점에서 변경을 확인하고, 지적마다 근거와 확인 방법을 제시하도록 구성했습니다.
- 지적 내용을 다시 확인해 타당한 의견만 반영하고, 수정 후 관련 검증과 남은 위험을 기록하도록 절차를 정했습니다.

달라진 점

실제 개발에서 팀 코드리뷰 전에 규칙 준수 여부를 추가로 점검하고 필요한 수정을 반영하는 데 사용했습니다.

검토를 요청하는 방식도 규칙으로 구성

변경 규모와 위험에 따라 Quick·Full·Skip을 선택하도록 했습니다. 아키텍처, 테스트, 빌드처럼 서로 다른 관점에서 검토하고, 단순 취향이나 추측 대신 파일과 변경 내용에 근거한 지적을 요구했습니다.

검토 의견을 그대로 적용하지 않음

관점별 검토가 끝나면 각 지적의 근거를 다시 확인하도록 했습니다. 타당한 지적은 수정하고 추측성 의견은 기각합니다. 수정 뒤에는 관련 테스트·빌드 등 필요한 검증을 진행하고, 채택·기각한 의견과 남은 위험을 기록하는 흐름으로 구성했습니다.

이 경험에서 배운 점

동료가 스킬로 AI의 작업 방식을 구성하는 모습을 보고, AI를 구현 요청뿐 아니라 사전 점검에도 활용할 수 있겠다고 생각했습니다. 직접 스킬을 만들고 적용하며 검토 관점과 의견을 수용할 기준을 정하는 방법을 배웠습니다.

03 · BOOSTCAMP · TEAM PROJECT

레츠톡

AI와 대화하며 작성하는 회고 서비스 · 2024.10–2024.12

부스트캠프 웹·모바일 9기 · iOS 4인 팀

App Store GitHub

회고 작성이 부담스러운 사람이 대화로 하루를 돌아볼 수 있도록 만든 앱입니다. 회고 데이터 저장과 CloudKit 동기화, 채팅 UI와 메시지 전송 · 실패 처리 로직을 담당했습니다.

SwiftUI UIKit Core Data CloudKit Combine Swift Concurrency

담당 범위

- 회고 데이터 저장과 CloudKit 동기화
- 채팅 UI와 메시지 전송·실패 처리 로직



회고 목록 대화형 회고

프로젝트 README에 보존된 당시 서비스 화면

CLOUDKIT SYNC

01 동기화된 최신 회고가 첫 실행 목록에 빠지던 문제

iCloud를 연동해 회고를 저장한 뒤 앱을 삭제하고 다시 설치하거나, 다른 기기에 새로 설치해 iCloud를 연동하면 최신 회고가 목록에 보이지 않았습니다. 앱을 다시 실행해야 나타나는 문제였습니다.

01 앱 시작: 로컬 회고 조회	02 CloudKit → Core Data import	03 import 성공 이벤트 수신	04 목록 재조회 → 최신 회고 표시

변경 전후 · 회고 목록 갱신 흐름

변경 전

로컬 목록 조회 → 늦게 import 완료

↓

저장소에 새 회고 반영 · 화면은 첫 조회 결과 유지

변경 후

로컬 목록 표시 → import 성공 이벤트

↓

목록 재조회 → 재실행 없이 최신 회고 표시

제가 한 작업

- Core Data의 SQLite 저장소를 직접 열어 최신 회고가 기기에 저장돼 있는지 확인했습니다.
- 화면의 첫 조회가 끝난 뒤 CloudKit 데이터가 로컬에 반영되고, 그 뒤 재조회가 없다는 순서를 원인으로 파악했습니다.
- NSPersistentCloudKitContainer의 import 성공 이벤트를 받아 회고 목록을 다시 조회하고 화면을 갱신하도록 연결했습니다.

달라진 점

앱 실행 직후에는 저장된 회고를 표시하고, CloudKit 데이터가 반영되면 목록을 갱신하도록 변경했습니다. 재실행 없이 최신 회고가 표시되는 것을 직접 확인했습니다.

저장 문제인지 화면 문제인지 먼저 구분

최신 데이터가 기기로 오지 않은 것인지, 저장됐는데 화면에 보이지 않는 것인지 구분할 필요가 있었습니다. SQLite 저장소를 직접 확인해 최신 회고가 이미 들어온 것을 보고 화면의 조회·갱신 과정으로 조사 범위를 좁혔습니다.

두 작업이 끝나는 시점의 차이

화면은 앱 시작 때 로컬 목록을 읽고, CloudKit import는 그 뒤에 끝날 수 있습니다. 저장소에 새로운 회고가 추가돼도 처음 읽은 화면 결과는 그대로였습니다. import가 성공적으로 끝났다는 이벤트를 화면 재조회의 기준으로 삼았습니다.

이 경험에서 배운 점

비동기로 데이터가 도착하는 시점과 화면이 조회하는 시점을 함께 확인하게 됐습니다. 실제 저장 상태를 확인하면 문제를 데이터 쪽과 화면 쪽으로 구분할 수 있다는 점도 배웠습니다.

04 · INTERNSHIP

엑소스피어랩스

macOS 고객지원 앱과 정책 변경 알림 연동 · 2023.12–2024.04

macOS 개발 인턴

사내 고객지원 담당자가 기존 보안 앱의 로그를 확인하고 정책을 제어하는 도구를 개발했습니다. macOS와 Windows를 공통으로 개발하는 방안을 검토하기 위한 Electron 연동 PoC도 맡았습니다.

Swift SwiftUI NSXPCConnection Electron FCM Vapor

담당 범위

- 고객지원 앱의 로그 조회·필터링·추출과 정책 on/off 연동
- Electron의 공통 알림 수신·전달 로직과 macOS local API 수신부

MACOS SUPPORT

01

고객지원 담당자가 로그와 정책을 확인하는 도구

고객 문의의 원인을 파악하기 위해 사내 고객지원 담당자가 보안 앱의 로그를 확인하고 정책을 제어할 수 있는 도구가 필요했습니다.

01 고객지원 화면에서 조회·제어	02 기존 보안 앱 기능 호출	03 로그 필터링·내보내기 / 정책 on·off

제가 한 작업

- SwiftUI로 로그 조회·필터링·파일 내보내기와 정책 활성화·비활성화 화면을 만들었습니다.
- NSXPCConnection 연결을 사용해 별도 고객지원 앱에서 기존 보안 앱의 기능을 호출하도록 했습니다.

달라진 점

사내 고객지원 담당자의 실제 고객지원 업무에 활용됐습니다.

기존 기능을 고객지원 업무에 연결

보안 앱이 제공하던 로그 추출과 정책 제어 기능을 고객지원용 화면에서 사용할 수 있게 하는 작업이었습니다. 지원 담당자가 문의 내용을 확인할 수 있도록 로그를 조회·필터링하고 외부 파일로 내보내는 동작을 구성했습니다.

화면과 기존 기능의 연결

기존 보안 앱의 로그 추출과 정책 제어 기능을 별도 고객지원 앱에서 호출하도록 연결했습니다. 담당자가 선택한 조건에 따라 로그를 확인하고, 정책을 켜고 끄는 동작을 수행하도록 화면을 구성했습니다.

PLATFORM INTEGRATION

02

Electron과 기존 보안 앱의 연결 가능성을 PoC로 확인

macOS와 Windows를 공통으로 개발하는 방안을 검토하며, Electron을 도입하기 전에 기존 보안 앱과 연결할 수 있는지 확인하는 PoC를 맡았습니다.

01 Electron: 정책 변경 알림 수신	02 딥 링크 / 로컬 API 전달	03 macOS Vapor 수신 / Windows 더미 호출

제가 한 작업

- Electron을 조사하고 정책 변경 알림을 기존 앱으로 전달하는 프로토타입을 구현했습니다.
- FCM 알림을 수신한 뒤 딥 링크와 로컬 API로 전달하는 연결 방식을 검토·구현했습니다.
- macOS는 Vapor 수신부를 만들고, Windows는 더미 API 호출로 확인했습니다.

달라진 점

macOS 요청 수신과 Windows 더미 API 호출을 확인했습니다. 인턴 종료 후 사수에게 개발한 Electron·Vapor 프로토타입이 실제로 활용된다는 이야기를 들었습니다.

전체 전환에 앞서 연결 가능성을 확인

네이티브 개발자 확보의 어려움과 두 OS의 공통 개발을 검토하는 상황에서 Electron 조사를 맡았습니다. Electron으로 화면을 만드는 것뿐 아니라, 기존 보안 앱에 알림과 요청을 전달할 수 있는지가 확인 대상이었습니다.

macOS와 Windows에서 확인한 동작

macOS에서는 Vapor 기반 로컬 HTTP 수신부를 구현해 요청 도달을 확인했습니다. Windows에서는 더미 API 호출을 확인했습니다.

BACKGROUND

학습과 경력의 기반

교육

- IT 연합 동아리 YAPP · 2026.05 – 2026.08
- 충남대학교 컴퓨터융합학부 · 2019.03 – 2025.02
- 네이버 부스트캠프 웹·모바일 9기 · 2024.07 – 2024.12
- CNU SW아카데미 2기 iOS · 2023.01 – 2023.04

자격 · 어학

- 정보처리기사 · 2025.12
- TOEIC Speaking IH(140) · 2025.06

moongoon.cnu@gmail.com GitHub · MoonGoon72